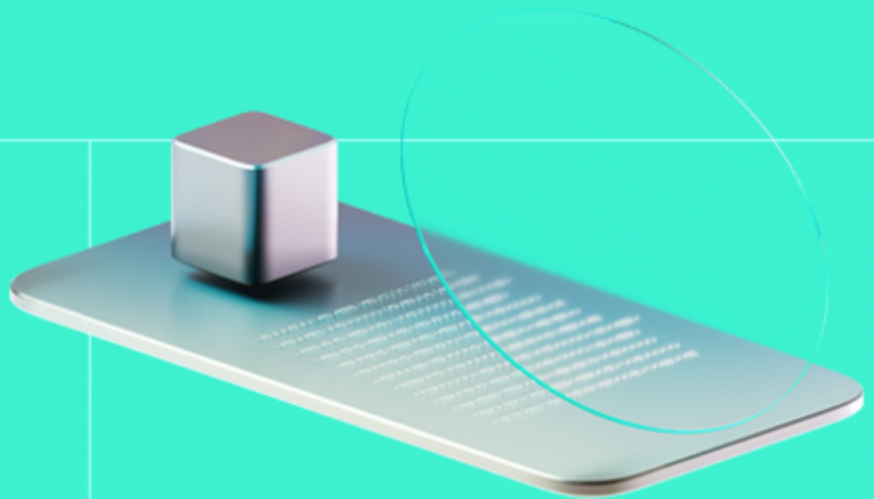




# Smart Contract Code Review And Security Analysis Report

**Customer:** Amara

**Date:** 09/10/2025



We express our gratitude to the Amara team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

AmaraStaking is a groundbreaking solution contract for the Amara token that offers fixed-APY reward tiers over predefined lock periods and applies a calibrated clawback to neutralize early exits. It uses epoch-aware accounting with a halving threshold to adjust rewards over time, supporting periodic claims and full withdrawals at lock expiry.

## Document

|             |                                                                                     |
|-------------|-------------------------------------------------------------------------------------|
| Name        | Smart Contract Code Review and Security Analysis Report for Amara                   |
| Audited By  | Kornel Światłowski, Panagiotis Konstantinidis                                       |
| Approved By | Ataberk Yavuzer                                                                     |
| Website     | <a href="https://www.amara.exchange/">https://www.amara.exchange/</a>               |
| Changelog   | 23/09/2025 - Preliminary Report                                                     |
|             | 09/10/2025 - Final Report                                                           |
| Platform    | EVM                                                                                 |
| Language    | Solidity                                                                            |
| Tags        | Staking                                                                             |
| Methodology | <a href="https://hackenio.cc/sc_methodology">https://hackenio.cc/sc_methodology</a> |

## Review Scope

|                    |                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Repository         | <a href="https://github.com/DioneProtocol/Amara-Staking">https://github.com/DioneProtocol/Amara-Staking</a> |
| Initial Commit     | 6c0507f                                                                                                     |
| Remediation Commit | af5ff72                                                                                                     |

## Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

|                |          |          |           |
|----------------|----------|----------|-----------|
| 15             | 13       | 1        | 1         |
| Total Findings | Resolved | Accepted | Mitigated |

### Findings by Severity

| Severity | Count |
|----------|-------|
| Critical | 0     |
| High     | 3     |
| Medium   | 7     |
| Low      | 2     |

| Vulnerability                                                                                                                                                  | Severity | Status    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------|
| <a href="#">F-2025-13049</a> - Double Division and Rounding in _calculateEarlyExitPenalty() Zeroes principalHaircut Nullifying the Intended Early-Exit Penalty | High     | Fixed     |
| <a href="#">F-2025-13051</a> - Principal Haircut Under-Applied Near Expiry Due to Month Flooring                                                               | High     | Fixed     |
| <a href="#">F-2025-13069</a> - Reward Calculation Reverts When Halving Timestamp Is After a Stake's Lock End Due to Underflow in _calculateReward Function     | High     | Fixed     |
| <a href="#">F-2025-13039</a> - Missing Reward Validation Can Lead to Blocks Stake Withdrawals                                                                  | Medium   | Accepted  |
| <a href="#">F-2025-13040</a> - Staking Tier Updates Alter Active Stake Rewards                                                                                 | Medium   | Fixed     |
| <a href="#">F-2025-13041</a> - Emergency Token Recovery Function Enables Asset Drain                                                                           | Medium   | Mitigated |
| <a href="#">F-2025-13087</a> - Incomplete Implementation of Emergency Withdrawal Logic                                                                         | Medium   | Fixed     |
| <a href="#">F-2025-13088</a> - Unrestricted Modification of Reward Halving Threshold and Timestamp                                                             | Medium   | Fixed     |
| <a href="#">F-2025-13144</a> - Overcounted Post-Halving Window in getTotalLockPeriodRewards() function Overstates Projected Rewards                            | Medium   | Fixed     |
| <a href="#">F-2025-13145</a> - Mixed-Unit Calculation of effectivePenaltyRate() Produces Invalid Penalty Rates                                                 | Medium   | Fixed     |
| <a href="#">F-2025-13043</a> - Use Ownable2Step instead of Ownable                                                                                             | Low      | Fixed     |

| Vulnerability                                                                                  | Severity | Status |
|------------------------------------------------------------------------------------------------|----------|--------|
| <a href="#">F-2025-13127</a> - Inaccurate Reward Indication in Public View Function            | Low      | Fixed  |
| <a href="#">F-2025-13038</a> - Floating Pragma                                                 | Info     | Fixed  |
| <a href="#">F-2025-13044</a> - Unused Error Definition                                         | Info     | Fixed  |
| <a href="#">F-2025-13118</a> - Redundant Variable Initialization Increases Deployment Gas Cost | Info     | Fixed  |

## Documentation quality

- Project overview is clear and well-structured. Core logic is explained with examples and tables.
- Run and deployment instructions are included.
- Technical specification is covered.
- Roles and access control are not explicitly documented.

## Code quality

- Development environment and scripts are well set up.
- Code follows solidity best practices.
- NatSpec is sufficient within the codebase.

## Test coverage

Code coverage of the project is **52.27%** (branch coverage).

- Test cases cover the main functional logic and core flows.
- A gas usage preview is included to provide an overview of method costs during execution.

# Table of Contents

|                                         |           |
|-----------------------------------------|-----------|
| <b>System Overview</b>                  | <b>7</b>  |
| Privileged Roles                        | 7         |
| <b>Potential Risks</b>                  | <b>8</b>  |
| <b>Findings</b>                         | <b>9</b>  |
| Vulnerability Details                   | 9         |
| Disclaimers                             | 51        |
| <b>Appendix 1. Definitions</b>          | <b>52</b> |
| Severities                              | 52        |
| Potential Risks                         | 52        |
| <b>Appendix 2. Scope</b>                | <b>53</b> |
| <b>Appendix 3. Additional Valuables</b> | <b>54</b> |

## System Overview

The **AmaraStaking** contract is a staking protocol built for the Amara token ecosystem. It enables users to lock their Amara tokens for fixed periods in exchange for rewards calculated at specific APY rates. Rewards are distributed over time and may be claimed periodically, subject to minimum claim intervals.

The system introduces a clawback mechanism designed to penalize early withdrawals. If a user exits before the lock period ends, both accrued rewards and a portion of the staked principal are subject to penalties. This ensures that short-term exits do not provide an advantage over longer-term commitments.

To preserve reward sustainability, the contract implements an epoch-aware halving mechanism. Once a specified reward distribution threshold is reached, APY rates for future rewards are automatically halved. The system splits calculations into pre-halving and post-halving segments to ensure fair allocation for all stakers.

The protocol also supports tier-based staking: each lock period (e.g., 2, 3, 6, 12, or 24 months) defines its own APY level and associated benefits. These tiers can be configured and updated by the contract owner, allowing flexibility in shaping staking incentives.

In addition, the contract provides functionality for:

- Tracking user stakes with unique IDs, including amount, lock period, and claim history.
- Claiming rewards periodically, provided sufficient reserve is available.
- Withdrawing staked tokens with or without penalties, depending on lock expiry.
- Administrative controls such as updating reward halving thresholds, modifying tier configurations, setting emergency withdrawal fees, pausing/unpausing operations, and recovering tokens mistakenly sent to the contract.

Overall, **AmaraStaking** enforces a structured reward system that balances long-term incentives with mechanisms to protect against abuse, while allowing controlled adaptability through owner-governed parameters.

## Privileged roles

The owner of the **AmaraStaking** contract holds full administrative authority over the protocol. This account is the only role permitted to perform sensitive operations, including:

- Updating staking tier configurations (APY, descriptions, and active status).
- Adjusting the reward halving threshold or manually triggering halving events.
- Setting the emergency withdrawal fee applied to early withdrawals.
- Pausing or unpausing the contract, thereby enabling or disabling all core user operations.
- Recovering any ERC-20 token from the contract.

## Potential Risks

- **Owner's Unrestricted Fund Control:** The owner can call `emergencyRecoverTokens()` to withdraw any ERC20 tokens from the contract, including reward reserves. This centralizes control of all funds in a single key and creates the risk of user funds being rendered inaccessible.
- **Owner-Defined Economics:** The owner can arbitrarily change APYs, staking tier configurations, reward halving thresholds, and emergency withdrawal fees. This introduces risks of sudden changes in expected yields or economic conditions for stakers.
- **Absence of Time-lock Mechanisms:** Critical operations such as halving triggers, APY changes, and fund recoveries can be executed instantly without a delay or review buffer, allowing harmful changes to take effect immediately.
- **Lack of Multi-signature Controls:** All privileged operations are executed solely by the owner, creating a single point of failure if the private key is compromised or misused.
- **Single Points of Failure:** The system is heavily centralized around the owner account. If the owner is compromised, malicious, or becomes inactive, the staking protocol could suffer permanent loss of funds or operational deadlock.
- **Reward Funding Dependency:** Rewards are not self-sustaining and must be manually funded by the owner. If insufficient rewards are supplied, users may be unable to claim their staking returns, undermining trust in the protocol.
- **Owner-Controlled Governance:** The owner unilaterally governs all aspects of the staking system, including fund movements, pause functionality, and reward logic, leaving no role for decentralized community participation or checks and balances.
- **Inability to Exit in Low-Reserve Scenarios:** Since rewards are paid directly from the contract balance, if insufficient tokens are available (due to withdrawal by the owner or poor funding), users may be unable to exit or receive their full reward entitlement.
- **Early Exit Haircut Risks:** The clawback system applies both reward penalties and principal haircuts. If misconfigured by the owner, these could become excessively punitive, discouraging participation or unfairly punishing users.

# Findings

## Vulnerability Details

### [F-2025-13049](#) - Double Division and Rounding in `_calculateEarlyExitPenalty()` Zeroes principalHaircut Nullifying the Intended Early-Exit Penalty - High

#### Description:

The `withdrawStake()` function relies on `_calculateEarlyExitPenalty()` to apply a **principal haircut** when users exit before their lock period ends. The haircut is intended to scale linearly at **2% per remaining month** (`PRINCIPAL_HAIRCUT_SCALE = 200`, i.e., 200 basis points).

However, the current implementation introduces a **rounding error** by dividing by `10,000` twice. First, the code divides `(remainingMonths * PRINCIPAL_HAIRCUT_SCALE)` by `10,000` when calculating the rate, which reduces the value to `0` in all practical cases (since always `remainingMonths * 200 < 10,000` for lock periods under 50 months). Then, the code divides again when applying the haircut, effectively compounding the error.

As a result, the computed haircut is **always zero**:

```
// PRINCIPAL_HAIRCUT_SCALE = 200 (2% per month)
uint256 remainingMonths = remainingLockTime / MONTH_DURATION;
uint256 principalHaircutRate = (remainingMonths * PRINCIPAL_HAIRCUT_SCALE) /
10000; //audit: loses precision
uint256 principalHaircut = (_stakeInfo.amount * principalHaircutRate) / 10000
; //audit: second division
```

This means that **no matter how much lock time is left, the penalty never applies**, when `remainingMonths` is less than 50, allowing users to withdraw their principal early without any deduction. This undermines the economic design of the staking system, since the **early-exit disincentive is effectively disabled** and users can game the system by withdrawing prematurely without cost.

#### Impact:

- Penalty mechanism is broken: No haircut is ever applied.
- Economic fairness compromised: Early exits become free, reducing fairness for long-term stakers.

- Incentive system invalidated: The intended deterrent against premature withdrawals is neutralized.

**Assets:**

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:****Fixed**

---

**Classification****Impact Rate:** 3/5**Likelihood Rate:** 5/5**Exploitability:** Independent**Complexity:** Simple**Severity:** **High**

---

**Recommendations****Remediation:**

Compute the haircut **once in basis points** and apply **a single division by 10,000** when multiplying by the principal. That preserves precision and yields the intended 2% per remaining month. Concretely, keep the rate in bps ( `principalHaircutRate = remainingMonths * PRINCIPAL_HAIRCUT_SCALE` ) and only divide when calculating the monetary amount, then enforce the existing 20% cap.

**Example fix:**

```
// 2% per remaining month, expressed in basis points
uint256 remainingMonths = remainingLockTime / MONTH_DURATION;
uint256 principalHaircutRate = remainingMonths * PRINCIPAL_HAIRCUT_SCALE; //
e.g., 2% * months (in bps)

// Apply once: divide by 10,000 when converting bps → fraction
uint256 principalHaircut = (_stakeInfo.amount * principalHaircutRate) / 10_000;

// Cap at 20% of principal (2000 bps)
uint256 maxHaircut = (_stakeInfo.amount * 2_000) / 10_000;
if (principalHaircut > maxHaircut) {
    principalHaircut = maxHaircut;
}
```

This preserves precision, correctly scales the haircut with remaining months, and enforces the intended economic penalty.

## Resolution:

**Fixed** at commit `af5ff72`. The principal haircut is now computed in basis points once ( `remainingMonths * PRINCIPAL_HAIRCUT_SCALE` ) and applied with a single `10,000` division when multiplying by principal, eliminating the double-division rounding bug that zeroed the haircut and restoring the intended 2%-per-remaining-month penalty.

## Evidences

### PoC: Double Division Bug Zeroes Principal Haircut on Early Exit

#### Reproduce:

#### PoC Steps:

- Deploy mocked ERC20 token and the `AmaraStaking` contract.
- Mint 10,000,000 tokens (18 decimal places) to `userA` and 2,000,000 tokens (18 decimal places) to the staking contract (reward fund).
- `userA` stakes 10,000,000 tokens with 18 decimal places for a 12-month lock (`stakeId = 0`).
- Advance blockchain time by 6 months (stake still locked).
- Retrieve stake info → elapsed: 6 months, remaining: 6 months.
- Call `calculateEarlyExitPenalty(userA, 0)` → computed `principalHaircut = 0`.
- `userA` calls `withdrawStake(0)` → withdraw succeeds.
- Actual haircut applied = 0 → full 10,000,000 principal returned + rewards.
- Console logs (all values shown in raw wei and human tokens, 18 decimal places)

As a result, the early-exit haircut mechanism is completely bypassed, undermining the intended disincentive for premature withdrawals.

#### PoC Code:

```
function test_PrincipalHaircut_IsZero_OnEarlyExit_After6Months() public {
    console2.log("NOTE: All token amounts use 18 decimals. We show both raw wei and human tokens.");
    _logAmount("Configured STAKE_AMOUNT: ", STAKE_AMOUNT);
    _logAmount("Configured REWARD_FUND: ", REWARD_FUND);

    // balances before
    _logBalances("Before stake", userA, address(staking));
    uint256 balBefore = token.balanceOf(userA);
```

```

// Stake for 12 months (stakeId = 0)
vm.prank(userA);
staking.stake(STAKE_AMOUNT, TWELVE_MONTHS);

// fetch stake start time to compute months later
AmaraStaking.Stake memory info = staking.getStakeInfo(userA, 0);
uint256 startTime = info.startTime;

// sanity: userA spent the stake amount
uint256 balAfterStake = token.balanceOf(userA);
assertEq(balAfterStake, balBefore - STAKE_AMOUNT, "userA should have
staked full amount");

_logBalances("After stake", userA, address(staking));

// advance block time by 6 months (still within lock)
uint256 monthsToProceed = 6;
vm.warp(block.timestamp + monthsToProceed * ONE_MONTH);

// compute months proceeded & remaining (integer months)
uint256 monthsProceeded = (block.timestamp - startTime) / ONE_MONTH;
uint256 monthsTotal = TWELVE_MONTHS;
uint256 monthsRemaining = monthsTotal > monthsProceeded ? (monthsTotal - monthsProceeded) : 0;

console2.log("----- TIME METRICS -----");
console2.log("Tier months (total):      ", monthsTotal);
console2.log("Months proceeded (elapsed): ", monthsProceeded);
console2.log("Months remaining:           ", monthsRemaining);

// Verify the penalty calculation shows zero principal haircut (bug)
AmaraStaking.PenaltyCalculation memory p = staking.calculateEarlyExit
Penalty(userA, 0);
_logAmount("Computed principalHaircut: ", p.principalHaircut);
console2.log("Principal haircut applied? ", p.principalHaircut > 0);
assertEq(p.principalHaircut, 0, "BUG: principalHaircut should be zero
due to double division");

// Withdraw stakeId 0
vm.prank(userA);
staking.withdrawStake(0);

// check balance after (principal returned in full + some rewards)
uint256 balAfterWithdraw = token.balanceOf(userA);
_logBalances("After withdraw", userA, address(staking));

// Must be at least full principal back (no haircut)
assertGe(balAfterWithdraw, balBefore, "balance should be >= initial balance");

```

```
// Right after staking, userA had (balBefore - STAKE_AMOUNT). After w
ithdraw, must be >= balBefore
assertGe(
    balAfterWithdraw - (balBefore - STAKE_AMOUNT),
    STAKE_AMOUNT,
    "principal must be fully returned"
);

// Final summary
uint256 userNetGain = balAfterWithdraw - balBefore; // >= 0 (principa
l + rewards)

console2.log("===== SUMMARY =====");
_logAmount("UserA net gain: ", userNetGain);
console2.log("Principal haircut proved ZERO? ", p.principalHaircut ==
0);

console2.log("Verdict: Early-exit principal haircut is NOT APPLIED du
e to double division.");
}
```

### Results:

[illegible]

```
18dp)
Contract balance: 7499000000000000000000000000 (raw wei) or 749900 (tokens, 18d
p)
===== SUMMARY =====
UserA net gain: 12501000000000000000000000000 (raw wei) or 1250100
(tokens, 18dp)
Principal haircut proved ZERO? true
Verdict: Early-exit principal haircut is NOT APPLIED due to double division
.

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.59ms (442.33µs
CPU time)

Ran 1 test suite in 114.65ms (1.59ms CPU time): 1 tests passed, 0 failed, 0 s
kipped (1 total tests)
```

## Files:

PrincipalHaircutBug.t.sol

## F-2025-13051 - Principal Haircut Under-Applied Near Expiry Due to Month Flooring - High

### Description:

In `_calculateEarlyExitPenalty()` function, the principal haircut is intended to scale with the remaining lock time (more time remaining = higher haircut). The function first derives remaining time, then converts it to months with integer division:

```
function _calculateEarlyExitPenalty(
    Stake memory _stakeInfo,
    uint256 _rewardAmount,
    uint256 _currentTime
) internal view returns (PenaltyCalculation memory) {
    uint256 timeStaked = _currentTime - _stakeInfo.startTime;
    uint256 remainingLockTime = _stakeInfo.lockPeriod - timeStaked;

    // ...

    uint256 rewardPenalty = (_rewardAmount * penaltyFraction) / 10000;

    // Principal haircut (intended: 2% per remaining month)
    uint256 remainingMonths = remainingLockTime / MONTH_DURATION; //audit: in
    tegeer division floors
    uint256 principalHaircutRate = (remainingMonths * PRINCIPAL_HAIRCUT_SCALE
    ) / 10000;
    uint256 principalHaircut = (_stakeInfo.amount * principalHaircutRate) / 1
    0000;
    // ...
}
```

Because `remainingMonths` is computed via `remainingLockTime / MONTH_DURATION`, any partial month is truncated to zero. For example, with ~25 days left, `remainingMonths` becomes `0`, with 2.5 months left, it counts only 2 months. So the haircut adds no principal penalty at the final month, even though meaningful time remains.

This creates a soft spot right before expiry: users can exit in the final partial month and incur a smaller-than-intended (or zero) haircut. Combined with the (fixed) per-month scale (2% via `PRINCIPAL_HAIRCUT_SCALE = 200`), the flooring undermines the graduated penalty curve and weakens the early-exit deterrent near lock end.

### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Fixed

## Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** High

## Recommendations

**Remediation:** Derive remaining months from the selected tier (in months) minus months actually staked (floored), and treat any remaining days in the current month as a whole month. Keep the rate in basis points and apply the existing 20% cap after computing the raw haircut.

### Example Fix:

```
// Count any remaining days as a full remaining month via tier math
uint256 timeStaked = _currentTime > _stakeInfo.startTime
    ? _currentTime - _stakeInfo.startTime
    : 0;

uint256 monthsStaked = timeStaked / MONTH_DURATION; // floor is fine here
uint256 remainingMonths = _stakeInfo.tier > monthsStaked
    ? (_stakeInfo.tier - monthsStaked) // if there are leftover days, this e
    ffectively counts them as a full month
    : 0;

// 2% per remaining month, expressed in basis points
uint256 principalHaircutRate = remainingMonths * PRINCIPAL_HAIRCUT_SCALE;

// Apply once: divide by 10,000 when converting bps → fraction
uint256 principalHaircut = (_stakeInfo.amount * principalHaircutRate) / 10_000;

// Cap at 20% of principal (2000 bps)
uint256 maxHaircut = (_stakeInfo.amount * 2_000) / 10_000;
if (principalHaircut > maxHaircut) {
    principalHaircut = maxHaircut;
}
```

This preserves precision, counts the last partial month, and restores the intended 2%-per-remaining-month penalty profile.

**Resolution:** **Fixed** at commit `af5ff72`. The principal haircut now derives `remainingMonths` from `tier - monthsStaked` (counting any partial month as a full month) and applies a 2%-per-remaining-month rate capped at 20%, closing the near-expiry soft spot created by integer division flooring and restoring the intended penalty functionality.

## Evidences

### PoC: Flooring of Remaining Months Nullifies Principal Haircut in Final Partial Month

#### Reproduce:

#### PoC Steps:

- Deploy mocked ERC20 token and the `AmaraStaking` contract.
- Mint 1,000 tokens to `userA` and 500 tokens to the staking contract for rewards.
- `userA` stakes 1,000 tokens for a 6-month lock period.
- Advance blockchain time by 5.5 months (leaving 0.5 month until expiry).
- Intended haircut = 2% (20 tokens), since any partial month should count as a full month.
- `userA` withdraws stake.
- Actual haircut applied = 0.
- Final balance = 1,000 principal + 68 reward tokens, proving no penalty was enforced.

As a result, users exiting in the final partial month avoid the intended haircut, weakening the early-exit deterrent and undermining the staking economics.

#### PoC Code:

```
function test_PrincipalHaircut_FlooringBug_LastHalfMonth() public {
    _logBalances("Before stake");

    // userA stakes for 6 months
    vm.prank(userA);
    staking.stake(STAKE_AMOUNT, SIX_MONTHS);

    AmaraStaking.Stake memory info = staking.getStakeInfo(userA, 0);
    uint256 startTime = info.startTime;

    _logBalances("After stake");

    // advance 5.5 months
    uint256 elapsed = (5 * ONE_MONTH) + (ONE_MONTH / 2);
```

```

vm.warp(startTime + elapsed);

// Intended haircut: count partial month as full → 1 month = 2%
uint256 intendedRateBps = PRINCIPAL_HAIRCUT_SCALE; // 200 bps = 2%
uint256 intendedHaircut = (STAKE_AMOUNT * intendedRateBps) / 10000;

// elapsed in months with one decimal (5.5)
uint256 elapsedTenths = (elapsed * 10) / ONE_MONTH;
console2.log("----- TIME METRICS -----");
console2.log("Total months:", SIX_MONTHS);
console2.log("Elapsed (months):", elapsedTenths / 10, ".", elapsedTenths
% 10);
console2.log("Intended haircut (count partial month as 1):", intendedHair
cut / 1e18, "tokens");

// withdraw to prove haircut is not applied
vm.prank(userA);
staking.withdrawStake(0);

_logBalances("After withdraw");

// Calculate actual reward received
uint256 finalBal = token.balanceOf(userA);
uint256 actualReward = finalBal - STAKE_AMOUNT; // balance - principal

console2.log("===== SUMMARY =====");
console2.log("UserA final balance:", finalBal / 1e18);
console2.log("Actual reward credited:", actualReward / 1e18, "tokens");
console2.log("Expected haircut:", intendedHaircut / 1e18, "tokens");
console2.log("BUG: Haircut was NOT applied, user kept full 1000 principal
+", actualReward / 1e18, "reward");
}

```

## Results:

```

Ran 1 test for test/PrincipalHaircutFlooring.t.sol:PrincipalHaircutFlooringTe
st
[PASS] test_PrincipalHaircut_FlooringBug_LastHalfMonth() (gas: 290431)
Logs:
--- Before stake ---
UserA balance: 1000
Contract balance: 500
--- After stake ---
UserA balance: 0
Contract balance: 1500
----- TIME METRICS -----
Total months: 6
Elapsed (months): 5 . 5
Intended haircut (count partial month as 1): 20 tokens

```

```
--- After withdraw ---
UserA balance: 1068
Contract balance: 431
===== SUMMARY =====
UserA final balance: 1068
Actual reward credited: 68 tokens
Expected haircut: 20 tokens
BUG: Haircut was NOT applied, user kept full 1000 principal + 68 reward

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.04ms (413.67µs
CPU time)

Ran 1 test suite in 5.82ms (1.04ms CPU time): 1 tests passed, 0 failed, 0 ski
pped (1 total tests)
```

## Files:

PrincipalHaircutFlooring.t.sol

## [F-2025-13069](#) - Reward Calculation Reverts When Halving Timestamp Is After a Stake's Lock End Due to Underflow in `_calculateReward` Function - High

**Description:** The contract's reward logic is designed to handle halving events by splitting reward accrual into two phases:

- Pre-halving rewards at the full APY.
- Post-halving rewards at half the APY.

This is implemented in `_calculateReward()` function, which, when block time has passed the halving, computes the post-halving portion by subtracting `halvingTimestamp` from the end of the current reward segment (`endTime`):

```
uint256 postHalvingStartTime = halvingTimestamp;
if (block.timestamp > postHalvingStartTime) {
    uint256 postHalvingTime;
    if (stakeInfo.lastClaimTime < postHalvingStartTime) {
        // Last claim before halving → accrue from halving
        uint256 endTime = block.timestamp > lockEndTime ? lockEndTime : block
        .timestamp;
        postHalvingTime = endTime - postHalvingStartTime; //audit: risk of un
        derflow
    }
    ...
}
```

The problem appears when a stake **ended before halving began** (`endTime < postHalvingStartTime`):

- Each stake has a `lockEndTime = startTime + lockPeriod`.
- If the protocol later records `halvingTimestamp` **after this** `lockEndTime`, then for that stake:
  - `endTime = lockEndTime` (because the lock is already finished).
  - `endTime < halvingTimestamp`.
- The subtraction `endTime - halvingTimestamp` then attempts to subtract a larger number from a smaller one.

In Solidity 0.8 and later, arithmetic underflow no longer silently wraps and it reverts. Thus, the calculation halts with an exception.

Example:

- A stake is created on Jan 1 with a 3-month lock (lock end: Mar 1).

- Six months later (Jun 1), a halving is triggered, setting `halvingTimestamp = Jun 1`.
- The user, whose stake already ended on Mar 1, now withdraws after the halving was set.
- `_calculateReward()` takes the post-halving branch because `block.timestamp > halvingTimestamp`.
- It computes `endTime = lockEndTime = Mar 1`, then attempts:
  - `postHalvingTime = endTime - halvingTimestamp // Mar 1 - Jun 1`
- Since Mar 1 is earlier than Jun 1, this subtraction is negative. In Solidity 0.8 and later, that causes an arithmetic underflow revert, so the entire call fails.

This means **any stake that ended before halving** becomes unrecoverable: reward queries, reward claims, and withdrawals will consistently revert. This effectively locks those funds and breaks frontend calls relying on `view` function.

As a result:

- Users with finished stakes cannot withdraw or claim rewards if halving was set later.
- UIs and integrations calling reward previews will fail entirely for these stakes.
- This creates a denial-of-service condition, blocking affected positions from normal lifecycle completion.

#### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 3/5         |
| Likelihood Rate: | 5/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | High        |

#### Recommendations

**Remediation:** Guard the post-halving window in order to never subtract a larger timestamp from a smaller one. Compute `endTime` as already done (`min(block.timestamp, lockEndTime)`), and then only accrue post-halving

time if halving actually started before that segment ends. Otherwise, set it to zero. This prevents underflow reverts for stakes that finished before the halving and preserves correct accrual across the boundary.

### Example Fix:

```
uint256 endTime = block.timestamp > lockEndTime ? lockEndTime : block.timestamp;

uint256 postHalvingTime;
if (stakeInfo.lastClaimTime < postHalvingStartTime) {
    // Halving may or may not overlap the segment; guard the subtraction
    if (endTime >= postHalvingStartTime) {
        postHalvingTime = endTime - postHalvingStartTime;
    } else {
        postHalvingTime = 0; // lock ended before halving → no post-halving rewards
    }
    ...
}
```

This change removes the underflow path, keeps rewards accurate when halving occurs after a stake's lock end, and allows `claimRewards()` and `withdrawStake()` functions to proceed normally.

### Resolution:

**Fixed** at commit `af5ff72`. The post-halving accrual now checks that `endTime >= halvingTimestamp` before subtracting, setting `postHalvingTime` to zero when a stake ended before halving—eliminating the underflow revert and restoring normal reward/withdraw flows and view calls.

## Evidences

### PoC: Halving Timestamp Set After Stake Expiry Causes Reward Underflow and DoS

#### Reproduce:

#### PoC Steps:

- Deploy `MockERC20` and `AmaraStaking` contracts.
- Mint 1,000 tokens to userA and 500 tokens to the staking contract as reward reserve.
- UserA stakes 1,000 tokens with a 3-month lock.
- Advance blockchain time to after the stake has ended (e.g., 3 months + 1 day).
- The contract owner triggers halving at this late timestamp (`halvingTimestamp > lockEnd`).

- Warp forward again so `block.timestamp > halvingTimestamp`, activating the post-halving branch.
- Attempting `calculateReward(userA, stakeId)` reverts with `panic(0x11)` due to underflow in `endTime - halvingTimestamp`.
- Attempting `withdrawStake(stakeId)` also reverts, locking the user's funds.

Any stake that ended before halving was set becomes unrecoverable. Both reward queries and withdrawals revert, leading to a denial-of-service for affected positions.

### Poc Code:

```
function test_HalvingAfterLockEnd_CausesUnderflow_DoS() public {
    // 1) user stakes for 3 months
    vm.prank(userA);
    staking.stake(STAKE_AMOUNT, THREE_MONTHS);

    AmaraStaking.Stake memory s = staking.getStakeInfo(userA, 0);
    uint256 startTime = s.startTime;
    uint256 lockEnd = s.startTime + s.lockPeriod;

    // 2) warp to AFTER the lock has ended (stake finished)
    vm.warp(lockEnd + 1);

    // 3) owner triggers halving NOW (which is after lock end)
    staking.triggerHalving(block.timestamp);
    uint256 halvingTs = block.timestamp;

    // 4) step forward so (block.timestamp > halvingTimestamp) branch is
    taken
    vm.warp(block.timestamp + 1);

    // Log the timeline as used by _calculateReward()
    _logTs("TIMELINE", startTime, lockEnd, halvingTs, block.timestamp);

    // Sanity: in this state, endTime = lockEnd, and lockEnd < halvingTs.
    // _calculateReward() will hit:
    //   postHalvingTime = endTime - halvingTimestamp;
    // which underflows and reverts with arithmetic panic(0x11)

    // 5) Any reward path should revert. First, the view helper:
    vm.expectRevert(stdError.arithmeticError); // panic code 0x11
    staking.calculateReward(userA, 0);

    // 6) Withdraw should also revert (it calls _calculateReward internal
    ly)
    vm.startPrank(userA);
    vm.expectRevert(stdError.arithmeticError);
```

```

    staking.withdrawStake(0);
    vm.stopPrank();

    console2.log("BUG: With halving set after lock end, reward math under
flows -> DoS on finished stake.");
}

```

## Results:

```

Ran 1 test for test/RewardHalvingUnderflow.t.sol:HalvingUnderflow_DoS_Test
[PASS] test_HalvingAfterLockEnd_CausesUnderflow_DoS() (gas: 273824)
Logs:
--- TIMELINE ---
startTime:          1
lockEnd:            7776001
halvingTimestamp:   7776002
block.timestamp:    7776003
endTime (used):     7776001
Relation: endTime < halvingTimestamp ? true
BUG: With halving set after lock end, reward math underflows -> DoS on finished stake.

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 903.71µs (199.63µs CPU time)

Ran 1 test suite in 5.55ms (903.71µs CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)

```

## Files:

RewardHalvingUnderflow.t.sol

## [F-2025-13039](#) - Missing Reward Validation Can Lead to Blocks Stake Withdrawals - Medium

### Description:

The Amara staking mechanism allows Amara ERC20 tokens to be staked to earn the same token as rewards under predefined tiers of 2, 3, 6, 12, and 24 months. The current implementation does not ensure that staked tokens and corresponding rewards can always be withdrawn. The following flaws contribute to this risk:

- **Missing reward validation in `stake()`**: The `stake()` function does not validate whether sufficient reward tokens exist in the contract at the time of staking. As a result, stakes can be created even when the available reward pool equals zero.
- **Withdrawal failures due to insufficient rewards**: The `withdrawStake()` function transfers both the staked principal and the calculated rewards. When the contract lacks sufficient reward tokens, the function reverts with the `InsufficientRewardReserve()` error. This prevents recovery of even the initially staked tokens.
- **Owner-controlled emergency recovery**: The `emergencyRecoverTokens()` function allows the contract owner to withdraw any ERC20 token, including the staking token. Even if enough reward tokens are present during `stake()` execution, this function can be used to drain staking or reward reserves, resulting in blocked withdrawals for all participants.

The combination of these flaws creates a staking system where assets can be locked temporarily. To unlock the initially staked tokens, additional tokens must be deposited manually to cover missing rewards. The lack of safeguards against depletion of reward reserves undermines reliability and introduces systemic risks to all participants.

### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

### Status:

Accepted

### Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 3/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

---

## Recommendations

**Remediation:** It is recommended to introduce strict validation in the `stake()` function to ensure that sufficient reward tokens exist before new stakes are accepted. Safeguards should be added to prevent the use of `emergencyRecoverTokens()` in a way that drains staking or reward reserves required for withdrawals.

**Resolution:** **Accepted.** The client is aware of the finding and stated:

We understand the issue and acknowledge that the current validation does not account for rewards already reserved for earlier stakers. For now, we plan to maintain a sufficiently funded reward balance to ensure that withdrawal failures do not occur in practice. Given this operational safeguard, we're comfortable accepting the current logic as-is and will monitor the reward pool to prevent any insufficiency.

## [F-2025-13040](#) - Staking Tier Updates Alter Active Stake Rewards - Medium

### Description:

The Amara staking mechanism allows Amara ERC20 tokens to be staked to earn the same token as rewards under predefined tiers of 2, 3, 6, 12, and 24 months. When a stake is created, the associated data is stored in the `Stake` struct.

```
struct Stake {  
    uint256 amount;  
    uint256 startTime;  
    uint256 lockPeriod;  
    uint256 lastClaimTime;  
    bool isWithdrawn;  
    uint256 tier;  
}
```

The contract also contains the `updateStakingTier()` function, which is restricted to the contract owner. This function enables modifications of staking tier parameters, such as APY basis points.

```
function updateStakingTier(uint256 _tier, uint256 _apyBasisPoints, string memory _description,  
                           bool _isActive) external onlyOwner {  
    stakingTiers[_tier] = StakingTier(  
        {apyBasisPoints : _apyBasisPoints, description : _description, isActive : _isActive});  
    emit StakingTierUpdated(_tier, _apyBasisPoints, _description, _isActive);  
}
```

The `Stake` struct does not store the APY of the given stake at the time of creation. Instead, it references the corresponding `StakingTier` struct stored in the `stakingTiers` mapping. Since `StakingTier` values are mutable, the APY can be updated via the `updateStakingTier()` function. This results in a scenario where active stakes are affected by retroactive changes in APY, undermining the reward guarantees initially expected during stake creation.

The absence of APY caching in the `Stake` struct allows unilateral modification of rewards for all active positions within a tier. This diminishes the reliability of the staking mechanism and can lower user promised rewards. The updated APY should be applied to stakes created after the update.

**Assets:**

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:****Fixed**

---

**Classification****Impact Rate:** 4/5**Likelihood Rate:** 4/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** **Medium**

---

**Recommendations**

**Remediation:** It is recommended to cache the APY value in the **Stake** struct at the time of stake creation and use this cached value in reward calculations. This ensures that the APY agreed upon during stake initialization remains constant throughout the staking period, preventing retroactive alterations of user rewards.

**Resolution:** **Fixed** at commit **af5ff72**. Each stake now snapshots its APY at creation ( **apyBasisPointsAtCreation** ) and all reward/penalty calculations read from this cached value, preventing retroactive tier updates from altering rewards on active positions.

## [F-2025-13041](#) - Emergency Token Recovery Function Enables Asset Drain - Medium

**Description:** The `emergencyRecoverTokens()` function allows the contract owner to withdraw any ERC20 token from the contract. This includes both user-staked tokens and tokens reserved for rewards.

```
function emergencyRecoverTokens(address _token, address _to, uint256 _amount)
external onlyOwner {
    require(_to != address(0), "Invalid recipient");
    IERC20(_token).transfer(_to, _amount);
}
```

If the contract owner's private key is compromised, a malicious actor with access to this key could withdraw all assets from the contract. The presence of such unrestricted withdrawal functionality effectively places full custody of user assets under a single private key, introducing a single point of failure.

**Assets:**

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Mitigated

### Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 3/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

**Remediation:** It is recommended to restrict the `emergencyRecoverTokens()` function to exclude user-staked tokens and tokens reserved for rewards so that only excessive tokens can be withdrawn.

## Resolution:

**Mitigated** at commit `af5ff72`. While `emergencyRecoverTokens()` now blocks draining below `totalActivePrincipal`, it still allows withdrawing the reward reserves, which will halt claims and withdrawals.

## [F-2025-13087](#) - Incomplete Implementation of Emergency Withdrawal Logic - Medium

**Description:** The `AmaraStaking` contract defines the `emergencyWithdrawalFee` variable, provides a setter function for this variable, and initializes it to 5% in the constructor. However, the current implementation does not include an emergency withdrawal mechanism, and the `emergencyWithdrawalFee` value is not applied in any calculations.

The presence of a partially implemented feature increases code complexity and can create confusion regarding available functionality. Users may assume the emergency withdrawal mechanism exists due to the presence of the fee variable and setter, although no such functionality is provided. Additionally, the absence of an emergency withdrawal mechanism prevents recovery of staked tokens when the contract is paused, which contradicts the expected design. The discrepancy between documented design goals and implemented logic reduces system predictability and increases the risk of operational issues during emergency conditions.

Furthermore, applying a withdrawal fee in emergency scenarios would result in unfair terms. Emergencies typically arise from incidents beyond user control, such as contract pauses or unexpected failures. Under such conditions, reducing the amount of tokens returned would penalize users for circumstances not caused by their actions.

**Assets:**

- `AmaraStaking.sol` [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Fixed

### Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

**Remediation:**

It is recommended to either fully implement the emergency withdrawal mechanism without applying a fee, or remove the `emergencyWithdrawalFee` variable and its setter to eliminate unused code and reduce confusion.

**Resolution:**

**Fixed** at commit `af5ff72`. A complete `emergencyWithdraw()` function was added. Users can exit via an emergency withdrawal and receive their principal minus the configurable `emergencyWithdrawalFee`, but no accrued rewards are paid on this path.

## [F-2025-13088](#) - Unrestricted Modification of Reward Halving Threshold and Timestamp - Medium

### Description:

The `AmaraStaking` contract contains a halving mechanism that reduces the APR once the total collected rewards reach the value stored in the `rewardHalvingThreshold` variable, which is set during contract deployment to `1_000_000e18`. On each reward claim, the contract checks if the collected rewards exceed the threshold. If the threshold is reached, the halving timestamp is recorded, and from that point rewards are distributed with the reduced APR. Rewards accumulated before the halving timestamp remain unaffected and continue to follow the normal APR. This ensures that halving occurs automatically when the predefined condition is met.

```
/**
 * @dev Updates the reward halving threshold (owner only)
 * @param _newThreshold New threshold value
 * @param _resetHalving Whether to reset the halving timestamp (useful for te
 * sting)
 */
function setRewardHalvingThreshold(uint256 _newThreshold, bool _resetHalving)
external onlyOwner {
    uint256 oldThreshold = rewardHalvingThreshold;
    rewardHalvingThreshold = _newThreshold;

    if (_resetHalving) {
        halvingTimestamp = 0;
    }

    emit RewardHalvingThresholdUpdated(oldThreshold, _newThreshold);
}

/**
 * @dev Manually triggers reward halving for testing purposes (owner only)
 * @param _timestamp Timestamp to set as halving timestamp
 */
function triggerHalving(uint256 _timestamp) external onlyOwner {
    require(halvingTimestamp == 0, "Halving already occurred");
    require(_timestamp <= block.timestamp, "Timestamp cannot be in the future
");

    halvingTimestamp = _timestamp;
    emit RewardHalvingTriggered(_timestamp, totalRewardsDistributed, rewardHa
```

```
lvingThreshold);  
}
```

However, two functions ( `setRewardHalvingThreshold()` and `triggerHalving()` ) restricted to the contract owner allow immediate modification of the halving threshold without restrictions and manual triggering of the halving mechanism with an arbitrary timestamp, including past timestamps. This enables unilateral modification of staking rules. If misused, these functions can reduce participant rewards by setting an artificially low threshold or by applying a halving timestamp in the past.

Furthermore, the ability to call these functions creates a frontrunning risk. A claim transaction with a high reward value could be frontrun by calling one of these functions, resulting in the claimant receiving only half of the expected rewards without prior notice or possibility of reaction.

The existence of unrestricted administrative control undermines the predictability of staking rewards, introduces governance-related risks, and allows potential manipulation of reward distribution.

#### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

#### Status:

Fixed

### Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 3/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

#### Remediation:

It is recommended to remove the functions that allow unrestricted modification of the reward halving threshold and timestamp to ensure predictable and immutable reward rules. This also prevents potential frontrunning scenarios that reduce rewards during the claiming process.

**Resolution:**

**Fixed** at commit `af5ff72`. The owner-only `setRewardHalvingThreshold()` and `triggerHalving()` were removed.

## [F-2025-13144](#) - Overcounted Post-Halving Window in getTotalLockPeriodRewards() function Overstates Projected Rewards - Medium

### Description:

The `getTotalLockPeriodRewards()` function is a view that returns the total rewards that would be earned during the entire lock period, splitting the calculation into a pre-halving portion (full APY) and a post-halving portion (halved APY) when `halvingTimestamp > 0`.

For the post-halving part it currently measures time from `halvingTimestamp` up to the stake's `lockEndTime`:

For the post-halving segment, the function starts counting from `halvingTimestamp` through to `lockEndTime`. It uses the interval `[halvingTimestamp, lockEndTime]` instead of the correct `[startTime, lockEndTime]` when the stake begins after the halving:

```
// post-halving portion (current)
if (halvingTimestamp < stakeInfo.startTime + stakeInfo.lockPeriod) {
    uint256 postHalvingStartTime = halvingTimestamp;
    uint256 lockEndTime = stakeInfo.startTime + stakeInfo.lockPeriod;

    if (postHalvingStartTime < lockEndTime) {
        uint256 postHalvingTime = lockEndTime - postHalvingStartTime; //audit
        : incorrect counting
        uint256 halvedAPY = apyBasisPoints / 2;
        postHalvingReward = (stakeInfo.amount * halvedAPY * postHalvingTime)
        / (10000 * YEAR_DURATION);
    }
}
```

Specifically:

- If a stake starts after halving (`stakeInfo.startTime > halvingTimestamp`), the real earning window should be `[startTime, lockEndTime]`.
- But the code measures from `halvingTimestamp` instead: `[halvingTimestamp, lockEndTime]`.
- That interval (`halvingTimestamp - lockEndTime`) is greater than `(stakeInfo.startTime - halvingTimestamp)` (which is the correct post-halving window), so the function overestimates total rewards for the lock period.

Example:

- `halvingTimestamp = day 0`

- `startTime = day 10`, `lockEndTime = day 40`
- Correct post-halving time should be: `lockEndTime - startTime => 40 - 10 = 30 days`
- The current code uses `lockEndTime - postHalvingStartTime => 40 - 0 = 40 days => +10 days of rewards that shouldn't exist.`

This affects only the projection returned by `getTotalLockPeriodRewards()` view function (UIs/analytics), not the actual payout path (`_calculateReward()` function), but it can mislead users about expected returns.

As a result, UI-facing projections will show inflated totals for stakes that begin after halving, potentially misleading users and dashboards about expected rewards.

#### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

#### Status:

Fixed

### Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 3/5         |
| <b>Likelihood Rate:</b> | 4/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Medium      |

### Recommendations

**Remediation:** Start the post-halving window at the later of `halvingTimestamp` and `stakeInfo.startTime`, so only time actually staked after halving accrues post-halving rewards.

#### Example Fix:

```
// corrected post-halving portion
if (halvingTimestamp < stakeInfo.startTime + stakeInfo.lockPeriod) {
    uint256 lockEndTime = stakeInfo.startTime + stakeInfo.lockPeriod;

    // start counting at max(halvingTimestamp, startTime)
    uint256 postHalvingStartTime = stakeInfo.startTime > halvingTimestamp
        ? stakeInfo.startTime
```

```

        : halvingTimestamp;

    if (postHalvingStartTime < lockEndTime) {
        uint256 postHalvingTime = lockEndTime - postHalvingStartTime;
        uint256 halvedAPY = apyBasisPoints / 2;
        postHalvingReward =
            (stakeInfo.amount * halvedAPY * postHalvingTime) / (10000 * YEAR_
DURATION);
    }
}

```

This mirrors the intended epoch logic and ensures projections only include time the stake was actually active after halving.

## Resolution:

**Fixed** at commit `af5ff72`. The post-halving window in `getTotalLockPeriodRewards()` now begins at `max(halvingTimestamp, startTime)`, eliminating overcounted days for stakes that start after halving and aligning UI projections with the actual payout logic.

## [F-2025-13145](#) - Mixed-Unit Calculation of effectivePenaltyRate() Produces Invalid Penalty Rates - Medium

### Description:

The `_calculateEarlyExitPenalty()` function is designed to calculate the penalty for withdrawing a stake before its lock period ends. Among its outputs is the `effectivePenaltyRate` variable, meant to represent the overall penalty rate by combining both the reward clawback (`penaltyFraction`) and the principal haircut (`principalHaircutRate`).

However, the implementation combines variables expressed in different units and applies an inconsistent scaling, by multiplication of `100`:

```
uint256 effectivePenaltyRate = (penaltyFraction + (principalHaircutRate * 100)) / 100;
```

The problem arises because the two terms are in different units:

- Case 1 — `penaltyFraction`:

This variable is expressed in basis points (bps), constrained by `BASE_EARLY_EXIT_PENALTY` and `MAX_EARLY_EXIT_PENALTY`. That means it ranges from 2500 bps (25%) up to 9500 bps (95%).

- Case 2 — `principalHaircutRate`:

In the current implementation it always evaluates to 0 due to a double division bug (already identified separately). But even if it were fixed to return a correct bps value, e.g.:

```
uint256 principalHaircutRate = remainingMonths * PRINCIPAL_HAIRCUT_SCALE;  
// 8 months * 200 bps = 1600 bps (16%)
```

the calculation would still be incorrect, since multiplying `1600` (bps) by `100`, during the calculation of `effectivePenaltyRate`, yields `160,000`, which is then added to `penaltyFraction` and divided by `100`:

```
penaltyFraction = 2500 (25%)  
principalHaircutRate = 1600 (16%)  
effectivePenaltyRate = (2500 + (1600 * 100)) / 100  
                     = (2500 + 160,000) / 100  
                     = 1625
```

Consequently the `effectivePenaltyRate` is interpreted as 1625%, when the true intent was ~41% (25% + 16%).

Thus, the formula either:

- Always returns the `penaltyFraction` alone (when haircut is zero, as currently), or
- Yields inflated/unbounded values when haircut is nonzero, because it multiplies one side by 100 without normalizing the other.

This mixing of bps and percent points invalidates the reported penalty rate and breaks economic logic.

As a result:

- Current state: Principal haircut contributes nothing, making `effectivePenaltyRate` equivalent to just `penaltyFraction`.
- If fixed later without addressing units: Penalty rate could surpass far beyond 100%, mispricing early exits.

This means that the penalty calculation becomes inconsistent and misleading, undermining the deterrent against early withdrawals and confusing users and UIs.

#### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

#### Status:

Fixed

### Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 3/5         |
| <b>Likelihood Rate:</b> | 3/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Medium      |

### Recommendations

**Remediation:** Normalize all penalty components to the same scale (basis points) and compute the total consistently. Avoid multiplying or dividing inconsistently across variables.

#### Example fix:

```
// Normalize both components to basis points
uint256 penaltyBps = penaltyFraction; // already in bps
uint256 haircutBps = remainingMonths * PRINCIPAL_HAIRCUT_SCALE; // e.g., 200
bps per month
```

```

// Cap haircut at 2000 bps (20%)
if (haircutBps > 2000) haircutBps = 2000;

// Combine safely
uint256 effectiveBps = penaltyBps + haircutBps;
if (effectiveBps > 10000) effectiveBps = 10000; // cap at 100%

// Return in bps for consistency
uint256 effectivePenaltyRate = effectiveBps;

```

This ensures both penalty types are consistently expressed in basis points, avoids arbitrary scaling, and enforces logical caps.

### Resolution:

**Fixed** at commit `af5ff72`. The mixed-unit calc was replaced with a bps-only approach: `effectivePenaltyRate` now sums `penaltyFraction` (bps) plus `principalHaircutRateBps`, and returns the result in bps, aligning units across components.

## F-2025-13043 - Use Ownable2Step instead of Ownable - Low

### Description:

The current implementation of the `AmaraStaking` contract is based on OpenZeppelin's standard `Ownable` contract for managing contract ownership. While functional, this approach poses a potential risk of accidental or unauthorized ownership transfers. For example, the current owner might mistakenly transfer ownership to an incorrect or unintended address. Additionally, once the transfer occurs, the new owner cannot reject or decline the ownership, which can lead to operational challenges.

Replacing `Ownable` with `Ownable2Step` from OpenZeppelin provides a two-step ownership transfer process, where the new owner must explicitly accept the transfer before it is finalized. This mechanism ensures that ownership transfers are intentional and mutually agreed upon, reducing the risk of accidental or malicious actions.

```
import "@openzeppelin/contracts/access/Ownable.sol";  
...  
contract AmaraStaking is Ownable, ReentrancyGuard, Pausable {
```

### Assets:

- `AmaraStaking.sol` [<https://github.com/DioneProtocol/Amara-Staking>]

### Status:

Fixed

## Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 2/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Low

## Recommendations

### Remediation:

Replace `Ownable` with `Ownable2Step` in the contracts to leverage the two-step ownership transfer mechanism.

## Resolution:

**Fixed** at commit `af5ff72`. Ownership control was migrated from `Ownable` to `Ownable2Step`, so transfers now require the recipient to explicitly call `acceptOwnership()` before completion, preventing accidental hand-offs and making admin changes safer and intentional.

## F-2025-13127 - Inaccurate Reward Indication in Public View Function - Low

### Description:

The `_calculateReward()` function is an internal function responsible for computing staking rewards. Rewards can only be claimed independently after one month, due to the condition enforced within the `claimRewards()` function.

```
function claimRewards(uint256 _stakeId) external nonReentrant whenNotPaused {  
    ...  
    uint256 timeSinceLastClaim = block.timestamp - stakeInfo.lastClaimTime;  
    if (timeSinceLastClaim < MIN_CLAIM_INTERVAL) revert ClaimIntervalTooShort()  
    ;  
  
    uint256 rewardAmount = _calculateReward(msg.sender, _stakeId);  
    if (rewardAmount == 0) revert NoRewardsAvailable();  
    ...  
}
```

A public view function `calculateReward()` is also available, which returns the current reward amount for a stake by internally invoking `_calculateReward()`.

```
function calculateReward(address _user, uint256 _stakeId) external view returns (uint256) {  
    return _calculateReward(_user, _stakeId);  
}
```

Before the first month of a given stake, these two functions behave inconsistently. The `claimRewards()` function reverts because the time-based requirement is not satisfied, while the `calculateReward()` function returns a non-zero amount, suggesting that rewards are available for claiming.

This discrepancy creates misleading output for users and external systems relying on the public `calculateReward()` view. It may cause confusion in off-chain integrations, interfaces, or monitoring tools, which could indicate that rewards are available when in fact they cannot yet be claimed.

### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 3/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Low

## Recommendations

**Remediation:** It is recommended to unify the behavior of both functions to ensure consistent reward logic. This can be achieved by either:

- Adding the same time-based restriction to the `calculateReward()` function, returning zero before the first month of staking.
- Removing the restriction from the `claimRewards()` function, ensuring that both functions follow identical rules.

**Resolution:** Fixed at commit `af5ff72`. The `calculateReward()` now mirrors `claimRewards()` by returning 0 when `timeSinceLastClaim < MIN_CLAIM_INTERVAL`, removing the misleading pre-interval preview.

## F-2025-13038 - Floating Pragma - Info

### Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.20`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

### Assets:

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

### Status:

Fixed

## Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

## Recommendations

### Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler

updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.22;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

#### **Resolution:**

**Fixed** at commit `af5ff72`. The floating pragma was locked to `pragma solidity 0.8.22;` to ensure builds use the exact tested compiler, preventing minor-version discrepancies and compiler-induced regressions.

## [F-2025-13044](#) - Unused Error Definition - Info

**Description:** The contract contains the `InvalidLockPeriod()` error definition that is not utilized within the codebase:

```
error InvalidLockPeriod();
```

The presence of redundant error definition increases deployment cost, introduces unnecessary complexity, and reduces clarity of the contract structure. Retaining unused elements can also mislead code reviewers regarding the actual behavior and intended functionalities of the system.

**Assets:**

- AmaraStaking.sol [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Delete the `InvalidLockPeriod()` error definition from the contract if it is not intended for future use.

**Resolution:** **Fixed** at commit `af5ff72`. The unused `InvalidLockPeriod()` error was removed.

## [F-2025-13118](#) - Redundant Variable Initialization Increases Deployment Gas Cost - Info

**Description:** In Solidity, state variables are automatically initialized to their default values during contract deployment. For instance, `uint256` defaults to `0`, and `bool` defaults to `false`. However, within the `AmaraStaking` contract, the variable `halvingTimestamp` is explicitly assigned to `uint256` default values.

```
constructor(address _amaraToken) Ownable(msg.sender) {  
  
    ...  
    halvingTimestamp = 0; // No halving has occurred yet  
  
    ...  
}
```

This explicit reassignment introduces unnecessary bytecode, slightly increasing the gas cost of deployment.

**Assets:**

- `AmaraStaking.sol` [<https://github.com/DioneProtocol/Amara-Staking>]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5  
**Likelihood Rate:** 5/5  
**Exploitability:** Independent  
**Complexity:** Simple  
**Severity:** Info

### Recommendations

**Remediation:** It is recommended to remove redundant assignments of default values to state variables during contract initialization. This change will reduce deployment gas costs and improve code conciseness without affecting contract behavior.

**Resolution:**

**Fixed** at commit `af5ff72`. Removed the redundant `halvingTimestamp = 0` assignment in the constructor, relying on Solidity's default initialization to trim bytecode and slightly reduce deployment gas.

## Disclaimers

### Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

### Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

## Appendix 1. Definitions

### Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

| Severity | Description                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.                                                                                       |
| High     | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.                                |
| Medium   | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low      | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.                                                                                           |

### Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

## Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details          |                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| Repository             | <a href="https://github.com/DioneProtocol/Amara-Staking">https://github.com/DioneProtocol/Amara-Staking</a> |
| Initial Commit         | 6c0507f8d2d48482d5c270564b4ecd3a98e9729d                                                                    |
| Remediation Commit     | af5ff72c2f41b26b69e5ae5afe2655c2af23b4b7                                                                    |
| Whitepaper             | N/A                                                                                                         |
| Requirements           | README.md                                                                                                   |
| Technical Requirements | README.md, CLAWBACK_SYSTEM.md                                                                               |

| Asset                                                                                                                            | Type           |
|----------------------------------------------------------------------------------------------------------------------------------|----------------|
| AmaraStaking.sol [ <a href="https://github.com/DioneProtocol/Amara-Staking">https://github.com/DioneProtocol/Amara-Staking</a> ] | Smart Contract |

## Appendix 3. Additional Valuables

### Verification of System Invariants

During the audit of Amara staking, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry's built-in fuzzing was employed to check how the protocol behaves under a wide range of randomized inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, 10 invariants were tested over 10,000 runs each. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

| Invariant                                                              | Test Result | Run Count |
|------------------------------------------------------------------------|-------------|-----------|
| Stake and Withdraw must preserve principal minus haircut               | Passed      | 10K+      |
| Claiming rewards before <code>MIN_CLAIM_INTERVAL</code> should revert  | Passed      | 10K+      |
| Claiming rewards after <code>MIN_CLAIM_INTERVAL</code> should succeed  | Passed      | 10K+      |
| Contract pause should block stake/claim/withdraw                       | Passed      | 10K+      |
| Max stake count per user must not exceed configured maximum            | Passed      | 10K+      |
| Inactive tiers should revert on stake                                  | Passed      | 10K+      |
| Emergency token recovery restricted to owner                           | Passed      | 10K+      |
| Claim must revert if reward reserve is insufficient                    | Passed      | 10K+      |
| Claim at exactly <code>MIN_CLAIM_INTERVAL</code> updates lastClaimTime | Passed      | 10K+      |
| Reward halving triggers automatically when threshold crossed           | Passed      | 10K+      |

### Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.